

FreeShap / LR / A1 의 collapse mechanism

repo 코드의 실제 수식 기준

iter_04 의 분석을 코드 수준으로 정리

1. 코드 수준의 setup 정리

먼저 우리가 무엇을 계산하는지 코드 그대로 정리한다. 모든 분석의 mechanism 해석이 이 정의 위에서 출발해야 한다.

1.1. eNTK (empirical Neural Tangent Kernel) 의 정의

task_imbalance_ntk.py 가 fine-tuned BERT model $f(x; \theta)$ 에서 NTK 를 계산한다 (Wang et al. 2024, FreeShap §3.1):

$$K(x, x') = \langle \nabla_{\theta} f(x; \theta), \nabla_{\theta} f(x'; \theta) \rangle$$

즉 두 sample 에 대한 model output 의 **parameter gradient** 의 **inner product**. 결과로 얻은 NTK pickle 의 shape 는 $(1, n + n_{\text{val}}, n)$ — 즉 train $n \times n$ block K_{train} 과 val $\times$ train block $K_{\text{train, val}}$ 만 포함 ($K_{\text{val, val}}$ 은 부재 — lens 2 분석의 fallback 사유).

1.2. KRR (Kernel Ridge Regression) 의 학습 식 —

EigenNTKRegression.forward (entks/ntk_regression.py:301-409)

TMC iteration 의 각 **subset** S 에 대해 KRR utility 를 다음 식으로 계산한다.

먼저 train sample 전체에 대해 **eigen 분해** 를 precompute:

$$K_{\text{train}} = \sum_{i=1}^n \lambda_i u_i u_i^{\top}$$

top-r mode 만 보존해서 **feature matrix** 구성 (ntk_regression.py:141-142):

$$\Phi_{\text{tr}} = U \sqrt{\lambda} \in \mathbb{R}^{n \times r}, \quad \Phi_{\text{te}} = K_{\text{test, train}} \frac{U}{\sqrt{\lambda}} \in \mathbb{R}^{n_{\text{val}} \times r}$$

여기서 u 는 top-r eigenvector 들의 column 행렬, $\text{sqrt}(\lambda)$ 는 diagonal scaling. LR 의 경우 $\text{idx} = \text{np.argsort}(\text{evals})[:, -1][:d]$ 로 λ 큰 순, A1 의 경우 (아래) **label-aware score** 순.

이제 subset S ($|S| = m$) 의 prediction (forward 메서드, line 322-325):

```
PhiS = self.phi_tr.index_select(0, idx_t) # m x r
yS = self.y[train_indices].to(dtype=torch.long)
YS = torch.nn.functional.one_hot(yS, num_classes=self.n_class).to(dtype=self.dtype)
```

즉 **raw** one-hot $Y_S \in \{0, 1\}^{m \times C}$ 사용. **centering 안 함**. 이 점이 우리 분석에서 중요 — KRR 의 **학습 target** 이 **raw** Y_S 이지 centered $\tilde{Y}_S = Y_S - P_S$ 가 아니다.

Ridge solve (line 385):

$$W = (\Phi_S^{\top} \Phi_S + \rho I)^{-1} \Phi_S^{\top} Y_S$$

prediction:

$$\hat{F}(\text{val}) = \Phi_{\text{te}} W = \Phi_{\text{te}} (\Phi_S^{\top} \Phi_S + \rho I)^{-1} \Phi_S^{\top} Y_S$$

이게 **intercept 0 형태** 이다. 이론적으로는 (위의 토론에서 보았듯) intercept 를 분리하면 $b = P_S$, $\alpha = (K_S + \rho I)^{-1} (Y_S - P_S)$ 가 되어 **centered residual** 만 kernel 이 학습하는 게 standard 인데, 이 코드는 **raw** Y_S 학습 + **intercept 0** 으로 **intercept 책임** 을 **kernel** 에 떠넘김. 의미: empty subset 의 prediction = $0 \in \mathbb{R}^C$, val 위에서 $\text{argmax}(\theta)$ = class 0 (default). 즉 **empty subset 의 default predictor** 는 “**항상 class 0 예측**”.

1.3. Shapley value 의 utility 와 marginal contribution

(dvutils/Data_Shapley.py:24-50, 81-122)

TMC Monte Carlo 의 각 **permutation** 에서 sample 을 한 개씩 더해가며 **marginal contribution** 측정:

```
new_v_entropy, new_acc = self.probe_model.kernel_regression_idx(...)
new_score = np.array([-new_v_entropy, new_acc])
marginal_contribs = new_score - old_score
```

여기서 new_acc 는 val 위의 **naive accuracy**. 즉:

$$U(S) = \text{acc}_{\text{naive}}(S) = \frac{1}{n_{\text{val}}} \sum_{j=1}^{n_{\text{val}}} \mathbb{1}[\arg \max_c \hat{F}_{j,c}(\text{val}) = y_{\text{val},j}]$$

(balanced accuracy 가 아닌 **naive** 를 사용한다는 것이 mechanism 의 핵심 — val majority class 의 weight 가 dominant.)

Shapley value:

$$\phi_i = \frac{1}{n!} \sum_{\pi \in S_n} [U(S_{< i}^{\pi} \cup \{i\}) - U(S_{< i}^{\pi})]$$

TMC sampling 으로 500 permutation 평균하여 추정. selected top-k% sample = `argsort(-phi.alt)[:k]`.

2. 핵심 질문 1 — 왜 imbalanced 세팅에서 한 쪽 label 만 선택되는가?

코드 식 위에서 직접 derive 한다.

2.1. Empty subset 의 KRR predictor

$|S| = 0$ 이면 Φ_S 가 빈 행렬 $\rightarrow W = 0 \rightarrow$ prediction $\hat{F} = 0$. `argmax(0) = 0` 이라 모든 val sample 을 class 0 으로 예측. 따라서:

$$U(\emptyset) = \text{acc}_{\text{naive}}(\emptyset) = P_{\text{val}}(0) = \text{val 의 class 0 비율}$$

valbal regime (val balanced) 에선 $U(\emptyset) \approx \frac{1}{C}$. trainbal regime (val cls85_05_05_05) 에선 $U(\emptyset) \approx 0.85$ (val majority class 0 비율).

2.2. 한 sample 을 추가했을 때 marginal contribution

sample i 가 추가될 때 KRR 가 그 sample 의 **input direction** 을 학습:

$$\hat{F}(\text{val}) = K_{\text{val}, x_i} \cdot (K(x_i, x_i) + \rho)^{-1} \cdot Y_i$$

(단일 sample subset 의 경우.) Y_i 가 **class c_i 의 one-hot** 이므로, prediction 의 각 row 의 **sign** 이 row j 의 $K(\text{val}_j, x_i)$ **sign** 에 의해 결정. NTK 가 일반적으로 양수 이므로:

$$\hat{F}_{j,c}(\text{val}) \propto K_{\text{val}_j, x_i} \cdot \mathbb{1}[c = c_i]$$

즉 **sample i 와 가까운 val sample** 들 의 prediction 이 class c_i 쪽으로 끌림.

따라서:

- val sample 중 원래 **class c_i** 이고 x_i 와 가까운 들 $\rightarrow$ 이미 (empty subset 에서) class 0 으로 예측되었던 것들 중 **class c_i 가 정답인 것** 이 정확 $\rightarrow$ marginal +
- val 의 다른 **class** 들 $\rightarrow$ prediction 이 class c_i 쪽으로 끌리지만 정답 아님 $\rightarrow$ 기존 정확 분류된 것 (class 0 이었다면) 일부 wrong $\rightarrow$ marginal -

2.3. Valbal regime 의 sample 선호

valbal (val balanced 50/50) 의 empty subset 시작점은 $U(\emptyset) \approx \frac{1}{2}$. 이는 val 의 class 0 (majority? minority? 보통 valbal 에서 class 0 = train majority class) 50% 가 정확.

train 의 class 0 sample (= train majority) 한 개 추가:

- prediction 이 class 0 쪽으로 더 강하게 끌림
- val 의 class 0 (이미 정확) 은 그대로 정확
- val 의 class 1 (원래 wrong) 은 여전히 **wrong** (오히려 더 강하게 class 0 으로 끌림)
- $\Delta U \approx 0$ 또는 약간 -

train 의 class 1 sample (= train minority) 한 개 추가:

- prediction 의 일부 row 가 **class 1** 쪽으로 끌림
- val 의 class 1 (원래 wrong) 중 x_i 와 가까운 것 일부가 **class 1** 정확
- val 의 class 0 (원래 정확) 은 x_i 와 가까운 것 일부 wrong (loss)
- 평균적으로 net + Δ — 왜? class 1 정답 비율 50% 가 증가할 여지 가 큰 반면 class 0 정답 50% 는 이미 **100% 정확 (default)**, 더 올라갈 수 없음 → 새 정답 획득 > 잃은 정답.

이 비대칭이 **train minority sample** 의 **marginal contribution** 가 평균적으로 + 인 spectral 근원. Shapley value 평균이 **minority sample > majority sample** → top-Shapley 가 minority dominate.

train 의 minority 가 10% (500 개) 인 valbal pos90 setting 에서 top-1% (50 개) 는 거의 **100% minority sample**. 이게 사용자가 관찰한 “한 쪽 label 만 선택” 의 정량 mechanism.

2.4. Trainbal regime 의 sample 선호

trainbal (train balanced cls33_33_33, val imbalanced cls85_05_05_05) 의 empty subset 시작점은 $U(\emptyset) = P_{\text{val}}(0) \approx 0.85$.

train 의 class 0 sample (= train balanced 의 한 third, val majority class 와 같은 **class**) 추가:

- prediction 이 class 0 쪽으로 끌림
- val 의 class 0 (이미 거의 100% 정확) 그대로 정확
- val 의 class 1, 2 (원래 wrong) 더 강하게 class 0 끌림 → 여전히 wrong
- $\Delta U \approx 0$

train 의 class 1 또는 2 sample (= val minority class) 추가:

- prediction 의 일부가 **class 1 또는 2** 쪽으로 끌림
- val 의 class 1, 2 (val 의 5% 씩) 중 일부가 정확 → +0.05 정도 증가
- val 의 class 0 (val 의 85%) 의 일부가 wrong → -0.05 정도 감소
- **net 0** 또는 약간 -

이 case 에서 **naive acc** 의 **marginal contribution** 가 **val majority class** 와 같은 **class** 의 train sample 에 더 큼. 왜? **val** 의 **majority class** 정확 분류 유지 가 정확도 maximize 의 직접 경로이기 때문.

따라서 trainbal 의 top-Shapley = **val majority class** 와 같은 **train class** 의 sample 들. trainbal cls90_05_05 의 경우 top-Shapley = **train** 의 **class 0 sample** 들 **100%** — 사용자가 관찰한 trainbal 의 1-class collapse 의 spectral 근원.

3. 핵심 질문 2 — 왜 그 selection 으로 KRR 재학습하면 prediction 이 망하는가?

이건 **centering** 의 비대칭 과 raw Y_S 학습 의 결합으로 설명된다.

3.1. Selected subset 의 class purity 가 만드는 trivial predictor

valbal 의 selected S = mostly class 1 (minority dominated) 인 경우. 다시 KRR utility 계산:

$$W = (\Phi_S^\top \Phi_S + \rho I)^{-1} \Phi_S^\top Y_S$$

만약 Y_S 의 모든 row 가 **class 1** 의 one-hot (0, 1) 이면:

- $\Phi_S^\top Y_S = \Phi_S^\top \cdot (0, 1) \times m$
- prediction $\hat{F}(\text{val}) = \Phi_{\text{te}} W$ 가 모든 **val sample** 에 대해 (**small, large**) 형태 — 즉 항상 **class 1** 예측

즉 **empty subset** 의 **default** 가 “**항상 class 0**” 였는데 **selected subset** 의 **result** 가 “**항상 class 1**” 로 **flip**. 둘 다 **constant predictor** — kernel 이 진짜 학습 한 게 아니라 **constant** 만 다름. naive acc 가 50% (class 1 의 val 비율) 로 회복 안 됨 (왜? **constant class 1 predictor** 이라 val class 0 의 50% 다 wrong).

수식으로 더 정확히: 만약 Y_S 의 모든 row 가 동일 vector v 이면 $Y_S = \mathbf{1}_m v^\top$ 이고:

$$W = (\Phi_S^\top \Phi_S + \rho I)^{-1} \Phi_S^\top \mathbf{1}_m v^\top = \left((\Phi_S^\top \Phi_S + \rho I)^{-1} \Phi_S^\top \mathbf{1}_m \right) v^\top$$

이건 **rank-1 행렬**. prediction $\hat{F}(\text{val}) = \Phi_{\text{te}} W$ 가 모든 val row 가 $v^\top$ 의 scalar multiple — 즉 모든 val sample 의 prediction 의 class 분포가 동일 → **argmax** 가 모든 val sample 에서 동일 class = $\arg \max(v)$.

즉 **constant class predictor**. balanced acc 는 $(\text{recall}\{\text{maj}\} + \text{recall}\{\text{min}\}) / 2 = (0 + 100) / 2 = 50\%$ 또는 $(100 + 0) / 2 = 50\%$ — **balanced metric** 의 **50% baseline** 으로 collapse.

3.2. Trainbal 의 collapse — class purity 의 반대 방향

trainbal 의 selected S = mostly class 0 (val majority 와 같은 class) 인 경우. 위와 동일한 논리로 prediction 이 **항상 class 0**. val 의 90% 가 class 0 이라 **naive acc 90%** 유지 (좋아 보임!) 인데 **balanced acc** 는 $(100 + 0 + 0) / 3 = 33\%$ (class 0 만 정확, class 1, 2 다 wrong).

이게 **FreeShap utility (naive acc)** 와 우리 평가 **metric (balanced acc)** 의 불일치 가 **trainbal** 의 **collapse** 를 **hide** 하다가 우리 분석에서 드러나게 만든 mechanism. naive acc 만 보면 “잘 작동” 처럼 보이는데 balanced acc 를 보면 무너졌음을 알 수 있다.

3.3. Centering 의 비대칭 다시 — 이론과 코드의 gap

위에서 $(Y - P_{\text{train}})$ centering 으로 KRR 을 derive 하면 학습 대상이 $\tilde{Y} = Y - P_{\text{train}}$ 이라 **prior** 가 잡지 못하는 **residual** 만 kernel 이 학습한다는 것을 확인했다 (이전 토론). 이 형태에서:

- majority sample $\rightarrow \tilde{Y}_i \approx 0 \rightarrow \alpha_i$ 작음 $\rightarrow$ KRR 학습 부담 적음
- minority sample $\rightarrow \tilde{Y}_i$ 큼 $\rightarrow \alpha_i$ 큼 $\rightarrow$ KRR 학습 부담 큼

코드는 이 centering 을 **KRR utility** 단계에서 안 한다 — $Y_S = \text{one_hot}(y_S)$ raw. 따라서:

- empty subset 의 default = $\arg \max(0) = 0$ (class 0) — **prior** 가 아니라 임의의 default
- 모든 marginal contribution 의 기준점이 “**always class 0**”

이 임의의 default 기준점 이 **class 0** 외의 sample 의 contribution 을 자동으로 inflate. valbal pos90 (class 0 majority) 의 minority sample 들이 **class 1** 이라 default 와 다름 $\rightarrow$ 큰 marginal. trainbal cls90_05_05 의 **val majority = class 0** 이라 train 의 class 0 sample 들이 default 와 같음 $\rightarrow$ 효과 작음, 반대로 train 의 class 1, 2 sample 들이 **default** 와 다르고 val 의 **minority direction** 알려줌 $\rightarrow$ 큰 marginal.

결국 두 regime 모두 “**empty subset** 의 **default class** 와 가장 다른 **class** 의 **train sample** 들” 이 **top-Shapley**. selected S 의 class purity 극단 $\rightarrow$ 위에서 본 **rank-1 W** $\rightarrow$ **constant predictor** collapse.

만약 코드가 **centering** 한 $\tilde{Y} = Y - P_{\text{train}}$ 을 KRR utility 의 학습 target 으로 썼다면 mechanism 이 덜 collapse 했을 가능성이 있다 (default 가 P_{train} 이 되어 진짜 **baseline** 위 **marginal** 측정). 하지만 이 **paper** 의 **baseline (FreeShap)** 은 raw Y_S 를 쓰고 있고, 따라서 위의 **collapse mechanism** 이 default 가 됨.

4. 핵심 질문 3 — A1 이 어떻게 회복하는가?

A1 의 monkey-patch (`task_imbalance_shapley.py:36-97`) 가 **EigenNTKRegression** 의 **mode selection** 만 바꾼다. KRR utility (forward) 자체는 동일.

4.1. A1 의 mode selection score — centered $\tilde{Y}$ 사용

코드 `task_imbalance_shapley.py:67-79`:

```

Y = np.zeros((n, C), dtype=np.float64)
Y[np.arange(n), y_int] = 1.0
Y_centered = Y - Y.mean(axis=0, keepdims=True)

coeffs = evecs.T @ Y_centered
c2 = (coeffs ** 2).sum(axis=1)

evals_pos = np.maximum(evals, 0.0)
rho_filter = float(self.lam)
filt = (evals_pos / (evals_pos + rho_filter)) ** 2
score = filt * c2

idx = np.argsort(score)[::-1][:d]

```

수식으로 쓰면:

$$\tilde{Y} = Y - P_{\text{train}} \quad (\text{train marginal centering, } P_{\text{train}} \text{ 은 } Y.\text{mean}(\text{axis}=0))$$

$$c_i^2 = \|u_i^\top \tilde{Y}\|_2^2 = \sum_{c=1}^C (u_i^\top \tilde{Y}_{:,c})^2$$

$$s_i = \left(\frac{\lambda_i}{\lambda_i + \rho} \right)^2 \cdot c_i^2$$

$$I_{A1} = \text{top-r by } s_i$$

LR 의 `idx = np.argsort(evals)[::-1][:d]` 는 λ 만 보고, A1 의 `idx = np.argsort(score)[::-1][:d]` 는 s_i 보고 mode 선택. 즉 **centered $\tilde{Y}$ 를 사용한 mode-wise scoring**. 다만 **centering 은 mode selection**에만 적용, **KRR utility**의 학습 **target** 은 여전히 raw Y_S .

4.2. A1 회복의 spectral 의미

위에서 (이전 토론) 본 **KRR prediction**의 **spectral 분해**:

$$K \cdot (K + \rho I)^{-1} \cdot \tilde{Y} = \sum_i \frac{\lambda_i}{\lambda_i + \rho} \cdot (u_i^\top \tilde{Y}) \cdot u_i$$

즉 mode i 의 prediction contribution의 **norm-squared** = $\left(\frac{\lambda_i}{\lambda_i + \rho} \right)^2 \cdot \|u_i^\top \tilde{Y}\|_2^2$ = 바로 s_i . A1의 score는 mode i 가 prediction에 기여하는 magnitude의 직접 측정.

따라서:

- LR: λ 만 보고 top mode 골라 → “ λ 큰데 $\tilde{Y}$ alignment 약한 mode” 잘못 포함 + “ λ 작지만 $\tilde{Y}$ alignment 강한 mode” 누락 → minority direction mode 누락 → KRR utility의 **empty subset default 보정 능력** 약화
- A1: $\lambda \times \|u_i^\top \tilde{Y}\|$ 모두 보고 top mode → **minority direction mode 보존** → KRR utility가 minority sample의 effect를 더 잘 잡아냄

A1의 KRR utility $U^{A1}(S)$ 의 marginal contribution dynamics:

- train minority sample 추가 시 → A1의 mode set가 minority direction 포함 → KRR prediction이 minority direction 학습 → val의 minority sample 정확 → marginal contribution +
- train majority sample 추가 시 → A1의 mode set가 majority direction도 포함 (λ 큰 mode들) → KRR prediction이 majority direction 학습 → val의 majority 정확 유지 + small + → marginal contribution +

두 class 모두 양수 **marginal**. A1의 Shapley value가 **class-balanced 분포**. top-k% selection이 **minority + majority mixed**.

4.3. Mixed selection의 KRR utility가 살아남는 이유

selected S = (e.g.) 70% minority + 30% majority 인 경우, Y_S 의 row들이 **동일 vector**가 아님 — rank-1 collapse 안 됨.

$$Y_S = [(0, 1); (0, 1); \dots; (1, 0); (1, 0); \dots]$$

$\Phi_S^\top Y_S$ 가 **non-trivial rank-C matrix**. $W = (\Phi_S^\top \Phi_S + \rho I)^{-1} \Phi_S^\top Y_S$ 도 **non-trivial**. prediction $\hat{F}(\text{val}) = \Phi_{\text{te}}^\top W$ 가 **val sample** 마다 다른 **class 분포** — 진짜 학습된 **predictor**. balanced acc 회복.

이게 A1 회복의 spectral mechanism: **mode selection** 의 **label-aware** 성 → **top-Shapley** 의 **class-mixed property** → **KRR utility** 의 **non-trivial W** → **balanced acc** 회복.

5. 사용자가 놓친 부분 정리

5.1. Centering 의 비대칭은 의도된 design 인가, 실수인가?

코드의 **KRR utility** 가 **raw Y_S** 인 건 FreeShap 의 원 **design** (Wang et al. 2024). 일반적인 KRR theory 의 **intercept** 분리 **standard** 와 다르다. 이 비대칭이:

- FreeShap 의 **baseline** 자체 가 **raw Y 학습 + intercept 0** 이라 **empty subset = constant zero predictor** 가 기준점 이 됨
- Shapley value 의 marginal contribution 이 **prior baseline** 이 아닌 **default class 0 baseline** 에 대한 측정이 됨
- imbalanced 세팅에서 **default** 와 다른 **class sample** 들이 자동으로 top-Shapley

이건 **paper** 의 **mechanism analysis** 에서 명시되지 않음 — 우리 분석의 **paper-ready insight** 가 될 수 있다. 만약 **raw Y 학습** 대신 **centered $\tilde{Y}$ 학습** 으로 utility 를 redefine 하면 collapse 가 덜 발생할 가능성. 단 이건 **FreeShap** 의 정의 변경 이라 우리 **paper** 의 새 **method** 가 될 수도.

5.2. Naive acc vs balanced acc 의 불일치

FreeShap 의 Shapley utility 는 **naive acc**. 우리 분석 **metric** 은 **balanced acc**. 이 불일치 때문에:

- trainbal regime 에서 FreeShap 의 top-Shapley 가 **val majority class** 의 **train sample** 만 골라도 **naive acc** 기준으로는 잘 작동 (90% 유지)
- 그러나 **balanced acc** 기준 으로 (val minority class 의 recall = 0) → 33% 로 collapse

paper 의 **balanced acc metric** 강조 가 **FreeShap** 의 **hidden flaw** 를 드러낸 trigger. 만약 FreeShap 의 utility 자체를 **balanced acc** 으로 redefine 하면 trainbal 의 collapse 가 덜 발생할 가능성. 이것도 **next iter** 의 후속 실험 으로 가능.

5.3. A1 의 회복의 한계 — 완전한 회복이 아닌 이유

trainbal 의 절반 case 에서 A1 도 random 한테 패배. 이유:

- A1 의 mode selection 도 **KRR utility** 의 **raw Y_S 학습** 의 **fundamental** 한계 는 못 메꿈
- A1 의 centered $\tilde{Y} = Y - P_{\text{train}}$ 사용은 **train marginal centering**. trainbal regime 에선 train 이 balanced 라 $P_{\text{train}} = (\frac{1}{3}, \frac{1}{3}, \frac{1}{3})$ — 모든 **class** 의 **minority direction** 강조 효과 약함
- 만약 A1 의 score 를 P_{val} **centering** 으로 (즉 $\tilde{Y}_{\text{via_val}} = Y - P_{\text{val}}$) 으로 한다면 **val task** 의 **minority direction** 을 직접 표적 — 더 강한 회복 가능성. 단 **val labels 모르니 unrealistic**. **unlabeled val** 의 **distribution** 만 안다면 가능 (BBSE-style)

이건 **iter_05** 의 후속 실험 으로 자연스럽게 이어진다.

6. 한 줄 요약

코드 수준 mechanism:

1. FreeShap 의 utility 는 **raw Y_S 학습 + intercept 0** (= empty subset prediction 0 = default class 0)
2. Shapley marginal contribution 이 “**default class 0 baseline 대비**” 의 형태 → **default** 와 다른 **class** 의 train sample 이 top-Shapley dominate (valbal: train minority, trainbal: val majority 와 같은 class 의 train sample)
3. Selected S 의 class purity 극단 → Y_S 가 **constant vector** (또는 1-2 class 만) → W 가 rank-1 → KRR prediction 이 **constant class predictor** → balanced acc 50% (또는 1/C) collapse

4. A1 은 **mode selection** 만 label-aware ($s_i = \left(\frac{\lambda_i}{\lambda_i + \rho}\right)^2 \cdot \|u_i^\top \tilde{Y}\|^2$, $\tilde{Y} = Y - P_{\text{train}}$) $\rightarrow$ KRR utility 가 minority direction 보존 $\rightarrow$ marginal contribution 이 **class-balanced** $\rightarrow$ top-k% selection 의 class mix $\rightarrow W$ non-trivial $\rightarrow$ balanced acc 회복
5. A1 의 회복은 **spectral mode 회복** 만, **Shapley utility** 의 raw Y 학습 **design** 의 hidden bias 는 못 메꿈 $\rightarrow$ trainbal 의 절반 case 에서 random 한테 패배